

检验鉴定模块 — D 同学技术报告

模块负责人: D 同学

所属项目: 版权保护平台 (copyright-protection-platform)

提交记录: e4ee651 — "完成检验鉴定模块: ELA篡改检测、水印校验增强、综合鉴定器重构"

涉及文件 (6个):

- modules/verification/__init__.py (2行)
- modules/verification/ela.py (184行)
- modules/verification/watermark_check.py (143行)
- modules/verification/validator.py (213行)
- routes/forensics.py (新增ELA图片路由, +24行)
- static/js/forensics.js (新增结论面板渲染, +67行)
- templates/forensics.html (新增ELA热力图展示区, +9行)

一、项目结构总览

```
copyright-protection-platform/
├── modules/
│   ├── forensics/                # C 同学: 取证打包模块
│   │   ├── __init__.py
│   │   ├── hasher.py            # SHA-256 / MD5 哈希计算
│   │   ├── metadata.py         # 取证元信息 JSON 生成
│   │   └── packager.py         # 证据 ZIP 打包
│   ├── steganography/          # 水印嵌入/提取底层
│   │   ├── __init__.py
│   │   ├── embed.py            # DCT 频域水印嵌入 (218行)
│   │   └── extract.py          # DCT 频域水印提取 (154行)
│   └── verification/           # ★ D 同学: 检验鉴定模块
│       ├── __init__.py         # 模块声明
│       ├── ela.py              # ★ ELA 误差水平分析 (184行)
│       ├── watermark_check.py  # ★ 水印校验比对 (143行)
│       └── validator.py        # ★ 综合鉴定器 (213行)
├── routes/
│   └── forensics.py            # Flask 路由 (调用鉴定器 + 提供ELA图片)
├── static/js/
│   └── forensics.js            # 前端交互 (上传 → 鉴定 → 打包)
└── templates/
    └── forensics.html          # 页面模板 (含ELA热力图展示区)
```

D 同学的模块 (modules/verification/) 是整个鉴定流程的核心, 负责:

- 接收嫌疑文件路径
- 依次执行 SHA-256 → ELA → 水印提取 → 水印比对
- 输出综合判定结论 + ELA 热力图 + 完整审计日志

二、ELA (Error Level Analysis) 误差水平分析

2.1 原理

ELA 是单张图片的自我检测，不需要任何参考图或原图。

将嫌疑图片以固定质量（Q=90）重新保存为 JPEG，比较"重压前"和"重压后"的像素差异。图片被篡改后，不同区域的 JPEG 压缩历史不一致——背景可能已被多次压缩趋于稳定，而贴入的素材来自其他来源、压缩次数不同。统一重压后，压缩历史不一致的区域会表现出异常高的误差水平。

与其他鉴定手段的对比：

手段	需要什么	比对目标
ELA	仅嫌疑图本身	图片自己重压后的版本
水印提取	仅嫌疑图本身	从 DCT 系数中解码水印文本
水印比对	嫌疑图 + 预期水印文字	提取的水印 vs 输入的版权信息

为什么压缩历史不同会暴露篡改：

区域	压缩历史	再次用 Q=90 重压后变化	ELA 差异值
背景（原图）	多次压缩，已稳定	变化很小	低（蓝/绿色）
贴入的素材	几乎没被压过	变化很大	高（红/白色）

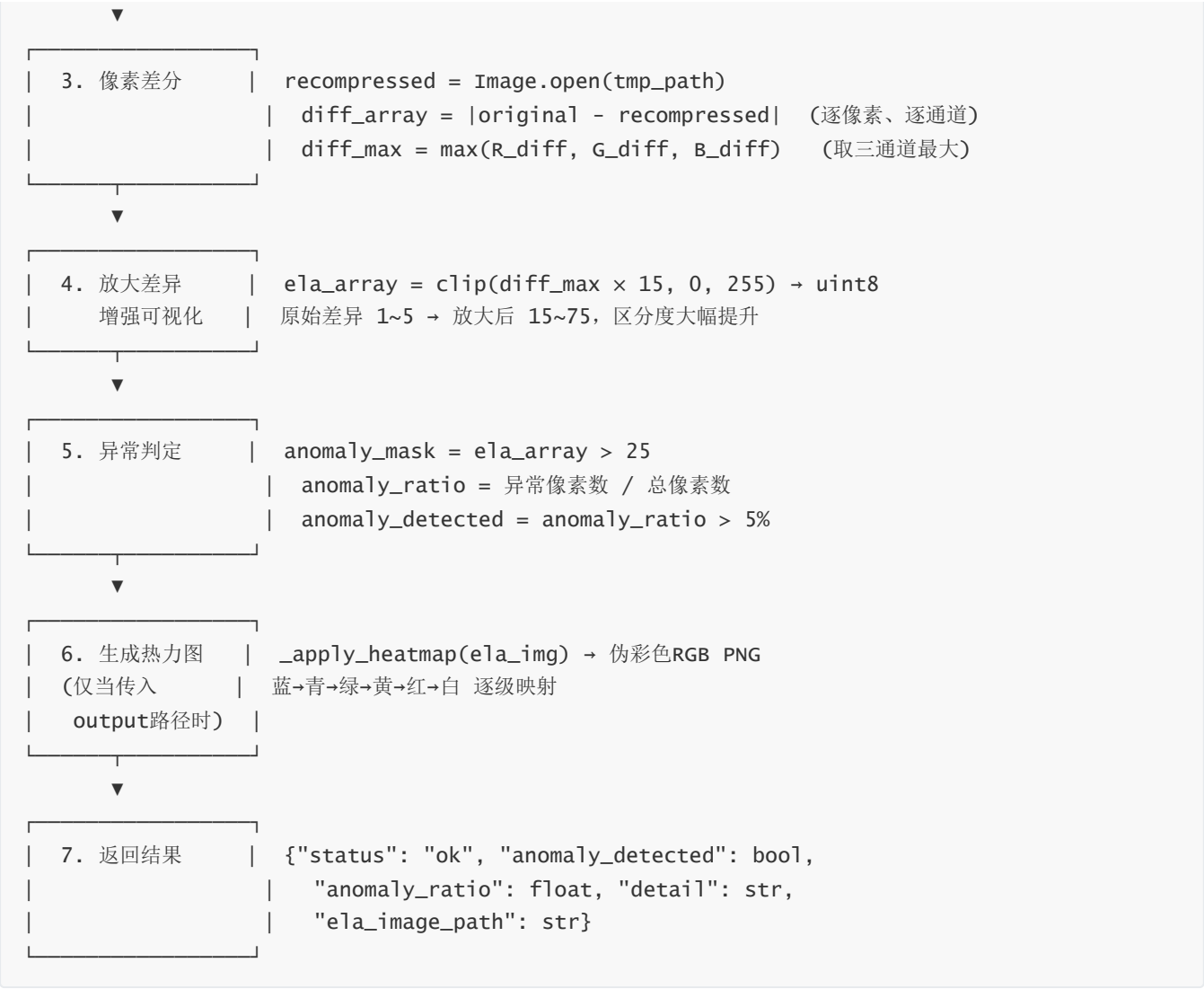
差异大的地方 = 和其他区域压缩历史不一致 = 疑似篡改区域。

2.2 核心参数（ela.py 第19-23行）

参数	值	说明
ELA_QUALITY	90	重保存 JPEG 的质量参数
ELA_SCALE_FACTOR	15	像素差异放大倍数（原始差异仅1~5，放大后可可视化）
ANOMALY_THRESHOLD	25	异常像素判定阈值（0-255亮度），超过即视为异常
ANOMALY_RATIO_THRESHOLD	0.05（5%）	异常像素占比超过此值即判定存在篡改

2.3 执行流程（ela_analysis() 函数，第26-116行）





2.4 热力图伪彩色映射 (_apply_heatmap() , 第119-184行)

不用 matplotlib (避免额外依赖) , 纯 numpy + PIL 手写 jet-like colormap, 分为5段线性插值:

差异值范围	颜色	含义
0.0 ~ 0.125	深蓝 → 蓝	低误差, 正常区域
0.125 ~ 0.375	蓝 → 青	较低误差
0.375 ~ 0.625	青 → 绿 → 黄	中等误差
0.625 ~ 0.875	黄 → 红	较高误差
0.875 ~ 1.0	红 → 白	极高误差, 疑似篡改核心区

```
# 分段1示例: 深蓝 → 蓝
mask1 = gray_array <= 0.125
t1 = gray_array[mask1] / 0.125
b[mask1] = 0.5 + 0.5 * t1 # B通道从0.5线性增到1.0
g[mask1] = 0.0           # G保持0
r[mask1] = 0.0           # R保持0
```

2.5 异常处理

- `FileNotFoundError`: 返回 `status=error`, 提示文件未找到
- `Exception`: 捕获所有其他异常, 返回 `status=error` + 异常信息
- `finally` 块: 始终清理临时 JPEG 文件, 避免磁盘残留

三、水印校验模块

3.1 模块职责 (`watermark_check.py`, 143行)

封装水印提取 + 比对逻辑, 提供统一的 `check_watermark()` 接口。

3.2 水印提取底层原理 (`steganography/extract.py`)

- 算法: 基于 DCT 频域的盲水印, 无需原图即可提取
- 嵌入位置: 图像 YCrCb 色彩空间的 Y (亮度) 通道, 每个 8×8 DCT 块的 (2,3) 和 (3,2) 中频系数对
- 编码方式: bit 1 使 `DCT[2,3] > DCT[3,2]`, bit 0 反之
- 抗裁剪机制:
 1. (3,1) 重复编码: 每个 bit 重复 3 次, 提取时多数表决
 2. 周期性循环嵌入: 数据帧 [同步头16bit + 长度16bit + 水印数据Nbit] 循环填入所有 DCT 块
 3. 滑动窗口搜索 16bit 同步头 [1,0,1,0,1,0,1,1,0,1,0,1,1,0,1,0]
 4. 多副本去重: 统计所有找到的水印文本, 取出现次数最多的作为最终结果

3.3 比对策略 (三种匹配方式, 按优先级依次尝试)

```
check_watermark(image_path, expected_watermark) → {
    extracted, match, detail, method, similarity
}
```

优先级	匹配方式	条件	similarity
1	精确匹配 <code>exact</code>	提取值 == 预期值 (strip后)	1.0
2a	子串包含 <code>substring</code>	预期值 ⊆ 提取值	len(预期)/len(提取)
2b	子串包含 <code>substring</code>	提取值 ⊆ 预期值	len(提取)/len(预期)
3	模糊匹配 <code>fuzzy</code>	Levenshtein 编辑距离 ≥ 70%	1 - 编辑距离/max_len

3.4 Levenshtein 编辑距离计算 (`_text_similarity()`, 第105-143行)

```
# 动态规划 + 滚动数组优化 (O(m×n) 时间, O(n) 空间)
prev = list(range(n + 1))
for i in range(1, m + 1):
    curr[0] = i
```

```

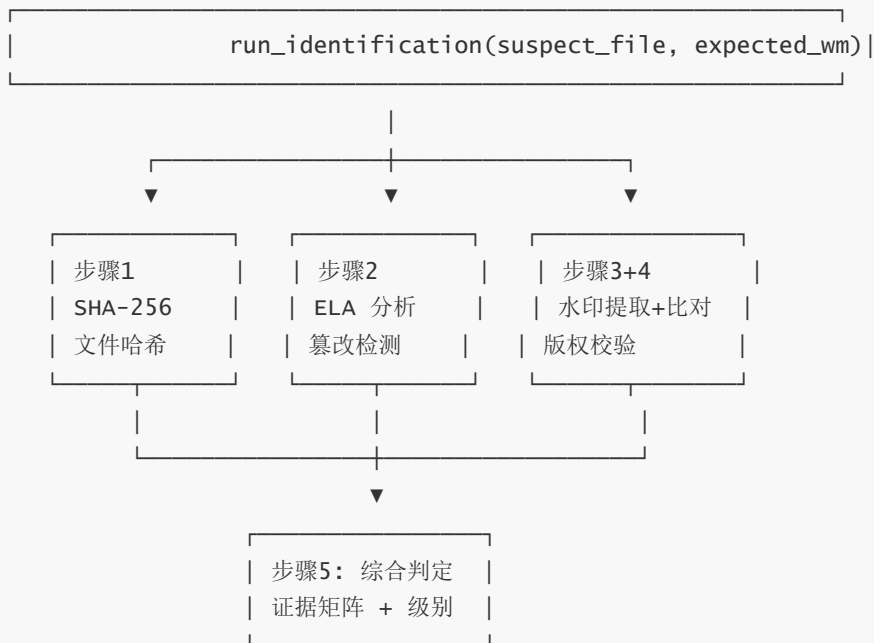
for j in range(1, n + 1):
    cost = 0 if s1[i-1] == s2[j-1] else 1
    curr[j] = min(
        prev[j] + 1,      # 删除
        curr[j-1] + 1,    # 插入
        prev[j-1] + cost  # 替换
    )
    prev, curr = curr, prev
distance = prev[n]
similarity = 1.0 - (distance / max_len)

```

阈值：相似度 ≥ 0.7 (70%) 即视为匹配成功。

四、综合鉴定器 (validator.py)

4.1 鉴定流程总览



4.2 每一步详情

步骤1：SHA-256 哈希计算

- 调用 C 同学的 `compute_sha256()` (`modules/forensics/haser.py`)
- 分块读取 (8KB 缓冲区)，支持大文件
- 输出：64位十六进制字符串

步骤2：ELA 篡改检测

- 调用 `ela_analysis(suspect_file, output_ela_image)`
- ELA 结果图输出路径: `data/reports/ela/ela_{hash前12位}_{timestamp}.png`
- 记录异常检测结果和异常像素占比

步骤3+4：水印提取与比对

- 调用 `check_watermark(suspect_file, expected_watermark)`
- 首先提取（不传预期值）获取水印内容
- 再传入预期值进行三种策略的匹配校验

4.3 综合判定矩阵

综合 ELA结果 和 水印比对结果 两个维度，生成四级结论：

水印提取	水印匹配	ELA异常	判定级别	判定意见
✔ 有	✔ 匹配	✔ 有	supported	技术鉴定支持侵权认定（双证据印证）
✔ 有	✔ 匹配	✘ 无	supported	技术鉴定支持侵权认定（水印证据）
✔ 有	✘ 不匹配	✔ 有	suspicious	存在嫌疑需复核（ELA异常+水印不匹配）
✔ 有	✘ 不匹配	✘ 无	inconclusive	证据不足（水印不匹配+ELA无异常）
✘ 无	—	✔ 有	suspicious	存在嫌疑需复核（ELA异常+无水印）
✘ 无	—	✘ 无	inconclusive	证据不足（无水印+ELA无异常）

4.4 四级结论定义

级别	常量	含义
supported	JUDGMENT_SUPPORTED	技术鉴定 支持 侵权认定，证据充分
suspicious	JUDGMENT_SUSPICIOUS	存在侵权嫌疑，需人工复核
inconclusive	JUDGMENT_INCONCLUSIVE	证据不足，无法认定
error	JUDGMENT_ERROR	鉴定过程异常

4.5 输出结构

```
{
  "success": True,
  "log": [
    {"step": "sha256", "status": "ok", "detail": "...", "timestamp": "..."},
    {"step": "ela", "status": "ok", "detail": "...", "timestamp": "..."},
    {"step": "watermark_extract", "status": "ok/fail", ...},
    {"step": "watermark_match", "status": "ok/fail", ...},
  ]
}
```

```
        {"step": "complete", "status": "ok", "detail": "最终判定", ...},
    ],
    "conclusion": {
        "file_sha256": "e2b359fa46e5...",
        "ela_result": "ELA检测发现异常: 异常像素占比 9.54%...",
        "ela_anomaly_detected": True,
        "ela_anomaly_ratio": 0.0954,
        "watermark_extracted": "张三123",
        "watermark_match": True,
        "watermark_match_method": "exact",
        "watermark_similarity": 1.0,
        "judgment_level": "supported",
        "final_judgment": "【技术鉴定支持侵权认定】嫌疑文件中检测到..."
    },
    "ela_image": "data/reports/ela/ela_e2b359fa46e5_1780073578.png"
}
```

五、与前端/路由的协作（D同学修改的前后端对接代码）

5.1 路由层（`routes/forensics.py` 新增部分）

- **第79-91行**: `identify()` 接口新增 ELA 图片 URL 生成逻辑, 从 `verification_result.ela_image` 构造可访问的图片地址
- **第138-151行**: 新增 `serve_ela_image()` 路由 `/forensics/api/forensics/ela-image/<session_id>/<filename>`, 直接返回 ELA 热力图 PNG 文件

5.2 前端展示（`forensics.js` 新增部分）

- **第137-191行**: 鉴定结论面板完整渲染, 包含:
 - 判定级别徽章（ 侵权认定/  存在嫌疑/  证据不足/  鉴定异常）
 - ELA 异常检测结果 + 异常像素占比
 - ELA 详细文字结果
 - 水印提取内容
 - 权属比对结果 + 匹配方式标签 + 相似度百分比
 - 最终结论（高亮显示）
- **第193-201行**: ELA 热力图展示区, 通过 `<img>` 标签加载 ELA 图片 URL

5.3 模板层（`forensics.html` 新增部分）

新增 ELA 热力图展示区域 `<div id="ela-image-area">` 及 `<img id="ela-image">` 元素, 在鉴定完成后展示伪彩色篡改检测结果图。

六、完整鉴定流程（用户视角）

用户操作

后端处理

技术产出

1. 上传嫌疑文件	→	保存到 <code>evidence_packages/</code> 计算 <code>SHA-256</code>	会话ID + 文件名
2. 填写元信息 点击"开始鉴定"	→	<code>create_metadata_json()</code>	<code>metadata.json</code>
3. 执行鉴定	→	<code>run_identification()</code> └ <code>compute_sha256()</code> SHA-256 哈希 └ <code>ela_analysis()</code> ELA 热力图 PNG └ 重压缩(JPEG Q=90) └ 像素差分 x15 └ 异常判定(>25, >5%) └ <code>jet-like</code> 伪彩色映射 └ <code>check_watermark()</code> └ <code>extract_watermark()</code> DCT盲水印提取 └ 滑动窗口+多数表决 └ 精确/子串/模糊匹配 <code>match_method</code> └ 综合判定矩阵 <code>judgment_level</code>	
4. 查看结果	→	前端渲染: └ 判定级别徽章 └ ELA 检测详情 + 热力图 └ 水印提取结果 + 比对详情 └ 最终结论文字	
5. 生成证据包	→	<code>build_evidence_package()</code>	<code>evidence_xxx.zip</code> └ 嫌疑文件 └ <code>metadata.json</code> └ <code>verification_report.json</code> └ <code>audit_log.json</code> └ <code>hashes.json</code>

七、技术亮点总结

亮点	说明
ELA 纯手写热力图	不依赖 matplotlib, 用 5 段线性插值手写 jet-like colormap, 零额外依赖
抗裁剪水印	周期性循环嵌入 + 滑动窗口同步头搜索 + 多副本投票去重, 裁剪后仍可提取
三级匹配策略	精确→子串→Levenshtein 模糊匹配, 兼顾精确性和容错性
滚动数组优化	Levenshtein 编辑距离用 O(n) 空间替代 O(m×n), 大文本友好
综合判定矩阵	6 种场景 × 4 级结论, 覆盖所有 ELA+水印 交叉组合
全流程可审计	每步记录 timestamp + status + detail, 最终写入 <code>audit_log.json</code>
异常安全	try/except/finally 覆盖所有步骤, 临时文件自动清理

